Universidad de los Andes Facultad de Ingeniería	**Ingeniería de Sistemas y Computación** **ISIS-1611: Inteligencia artificial** Semestre: 2026-10 Créditos: 3	Accredited ABET Engineering Accreditation Commission

Taller 4: Planificación Automatizada

Semanas después del devastador terremoto en Chapinero y de las complejas misiones de drones en el Amazonas, la Unidad Nacional para la Gestión del Riesgo y Desastres (UNGRD) enfrenta un nuevo desafío: coordinar la fase de reconstrucción y atención humanitaria en múltiples zonas afectadas de Bogotá. Ya no basta con encontrar un camino o evitar adversarios; ahora es necesario planificar secuencias completas de acciones interdependientes que permitan al robot SAR rescatar sobrevivientes, transportar suministros médicos y establecer puestos de atención, todo ello respetando un orden lógico y eficiente.

La "Operación Fénix" requiere de un planeador automatizado: un sistema capaz de razonar sobre el estado actual del mundo, las acciones disponibles y los objetivos a alcanzar, para generar un plan de acción válido y óptimo. La representación del problema usa un lenguaje formal similar a PDDL (Planning Domain Definition Language), que permite describir estados, acciones con precondiciones y efectos, y objetivos de manera estructurada e independiente del dominio específico.

Su responsabilidad es diseñar e implementar el motor de planificación que permita al robot SAR ejecutar misiones complejas de rescate: desde la búsqueda hacia adelante en el espacio de estados (forward planning), pasando por la búsqueda regresiva desde el objetivo (backward planning), hasta el uso de heurísticas de planificación y la descomposición jerárquica de tareas (HTN). Cada técnica tiene sus propias fortalezas, y usted aprenderá a identificar cuándo aplicar cada una.

Objetivos de aprendizaje

Al completar este taller, el estudiante será capaz de:

1. Modelar problemas de planificación clásica usando un lenguaje formal tipo PDDL: definir estados, esquemas de acción (precondiciones y efectos), y objetivos.
2. Implementar búsqueda hacia adelante (forward planning) en el espacio de estados para generar planes válidos.
3. Implementar búsqueda regresiva (backward planning / regression search) a partir del estado objetivo.
4. Diseñar e implementar heurísticas de planificación admisibles: ignorar precondiciones e ignorar listas de borrado (ignore-delete-lists).
5. Implementar un planificador jerárquico (HTN) con acciones de alto nivel (HLAs) y sus refinamientos.
6. Analizar el trade-off entre completitud, optimalidad y eficiencia computacional en los distintos enfoques de planificación.

Escenario del Problema

El robot SAR de la UNGRD ha sido adaptado para la fase de reconstrucción. Ahora opera en un entorno modelado como una cuadrícula que representa un sector de Bogotá en recuperación. Además de navegar el terreno, el robot debe ejecutar acciones estructuradas: recoger suministros, transportarlos, establecer puestos médicos y rescatar a los sobrevivientes en un orden correcto.

El mundo se representa mediante un conjunto de fluentes (proposiciones que describen el estado actual): posición del robot, ubicación de objetos, condiciones de las instalaciones, y el estado de cada sobreviviente. A diferencia de los talleres anteriores, la solución no es simplemente un camino en el mapa, sino una secuencia de acciones simbólicas que transforman el estado del mundo hasta alcanzar el objetivo.

Los elementos del entorno son los siguientes:

Símbolo	Tipo de Elemento	Descripción
`R`	**Robot SAR**	El robot de búsqueda y rescate reutilizado como plataforma de planificación. Posición inicial en el mapa.
`S`	**Sobreviviente / Paciente**	Persona que requiere atención. Para ser rescatada, debe ser encontrada y transportada al puesto médico.
`M`	**Puesto Médico**	Instalación donde se atienden los pacientes. Es el destino final de cada operación de rescate.
`T`	**Suministro Médico**	Caja con materiales (medicamentos, equipo quirúrgico). Debe ser llevada al puesto médico antes de atender pacientes.
`%`	**Muro / Obstáculo**	Estructura impasable. El robot no puede ocupar esta celda.
`.`	**Piso libre**	Celda transitable sin costo adicional.

Las acciones disponibles para el robot SAR son las siguientes. Su motor de planificación debe usar estas acciones con sus precondiciones y efectos para generar planes:

Acción	Parámetros	Precondiciones	Efectos

`Move(r, from, to)`	`r: robot`
	`from: celda`
	`to: celda adyacente`
		`At(r, from)`
		`Adjacent(from, to)`
		`Free(to)`
			`At(r, to)`
			`¬At(r, from)`

`PickUp(r, obj, loc)`	`r: robot`
	`obj: objeto`
	`loc: celda`
		`At(r, loc)`
		`At(obj, loc)`
		`HandsFree(r)`
		`Pickable(obj)`
			`Holding(r, obj)`
			`¬At(obj, loc)`
			`¬HandsFree(r)`

`PutDown(r, obj, loc)`	`r: robot`
	`obj: objeto`
	`loc: celda`
		`At(r, loc)`
		`Holding(r, obj)`
			`At(obj, loc)`
			`¬Holding(r, obj)`
			`HandsFree(r)`

`Rescue(r, p, loc)`	`r: robot`
	`p: paciente`
	`loc: puesto médico`
		`At(r, loc)`
		`At(p, loc)`
		`MedicalPost(loc)`
		`SuppliesReady(loc)`
			`Rescued(p)`
			`¬At(p, loc)`

`SetupSupplies(r, s, loc)`	`r: robot`
	`s: suministros`
	`loc: puesto médico`
		`At(r, loc)`
		`At(s, loc)`
		`MedicalPost(loc)`
		`Holding(r, s)`
			`SuppliesReady(loc)`
			`¬Holding(r, s)`
			`HandsFree(r)`

Nota: Las celdas del mapa definen la topología del entorno. Las relaciones Adjacent(a, b) se generan automáticamente para celdas horizontalmente o verticalmente contiguas que no sean muros. La relación Free(c) indica que la celda c no está ocupada por el robot. El predicado Pickable(obj) es verdadero para objetos T (suministros) y S (pacientes cuando están en camilla).

Estructura del Proyecto

Usted recibe un esqueleto base funcional que incluye la infraestructura de representación PDDL, la simulación del mundo y la visualización. Su responsabilidad es completar los algoritmos de planificación en los archivos designados:

- Módulo planning/:
  - `planner.py:` Contiene las implementaciones de los algoritmos de planificación forwardSearch, backwardSearch y aStarPlanner. Aquí escribirá la lógica principal de búsqueda de planes.
  - `heuristics.py:` Define las heurísticas de planificación (ignorePreconditionsHeuristic, ignoreDeleteListsHeuristic). Guía el planificador A* hacia el objetivo de forma eficiente.
  - `htn.py:` Implementación del planificador HTN. Define las acciones de alto nivel (HLAs), sus refinamientos, y el algoritmo de búsqueda jerárquica.
  - `pddl.py:` Define las clases State, Action, Problem y PDDLWorld. Representa el estado del mundo como un conjunto de fluentes, y las acciones como esquemas con precondiciones (add list y delete list).
  - `problems.py:` Define SimpleRescueProblem y MultiRescueProblem. Especifica el estado inicial, los estados objetivo y la función de sucesores para cada tipo de misión.
  - `utils.py:` Proporciona estructuras de datos (Stack, Queue, PriorityQueue) y funciones auxiliares para la búsqueda.
- Módulo world/:
  - `game.py:` Coordina el estado del mundo, el movimiento del robot, y la ejecución del plan generado por el planeador.
  - `rescue_layout.py:` Carga los archivos .lay y genera el estado inicial del mundo PDDL a partir del mapa.
  - `rescue_rules.py:` Define qué acciones son aplicables en cada estado y calcula los estados resultantes.
- Módulo view/: Contiene módulos de visualización gráfica y textual de la ejecución del plan.
- Carpeta layouts/: Contiene archivos .lay con mapas de prueba organizados en subcarpetas:
  - layouts/simple/: Mapas con un solo objetivo de rescate.
  - layouts/multi/: Mapas con múltiples sobrevivientes y suministros.
  - layouts/htn/: Mapas diseñados para probar la planificación jerárquica.

Ejecución

El programa se ejecuta con el siguiente formato general:

```
python main.py -p PROBLEM -f PLANNER [-h HEURISTIC] [-m] -l LAYOUT [OPCIONES]
```

Las opciones disponibles se describen a continuación:

Opción	Descripción	Default	Ejemplo
-p	Especifica el problema a resolver. Puede ser SimpleRescueProblem o MultiRescueProblem.	Ninguno	`-p SimpleRescueProblem`
-f	Especifica el algoritmo de planificación: forwardSearch, backwardSearch o aStarPlanner.	Ninguno	`-f forwardSearch`
-h	Heurística para A* (obligatoria si se usa aStarPlanner): ignorePreconditions o ignoreDeleteLists.	Ninguno	`-h ignorePreconditions`
-l	Archivo de mapa a cargar (sin extensión .lay). Define el entorno de la misión.	Ninguno	`-l tinyBase`

Opción	Descripción	Default	Ejemplo
`-m`	Activa el modo HTN (Hierarchical Task Network) en lugar de planificación clásica.	Desactivado	`-m`
`-t`	Activa modo texto. Útil si tkinter no está disponible.	Desactivado	`-t`
`-q`	Salida mínima en consola, sin gráficos.	Desactivado	`-q`
`-z`	Escala el tamaño de la ventana gráfica.	1.0	`-z 2.0`
`-x`	Tiempo de espera en segundos entre pasos de visualización.	0.1	`-x 0.5`
`--help`	Muestra la ayuda con todas las opciones disponibles.	—	`--help`

Puede verificar que el proyecto se ejecuta correctamente con el siguiente ejemplo de referencia:

```
python main.py -p SimpleRescueProblem -f tinyBaseSearch -l tinyBase -x 0.5 -z 2.0
```

Tip: Analice el código de `tinyBaseSearch` en `planning/planner.py` y el layout de `tinyBase`. Note que sus algoritmos de planificación deben retornar una secuencia de acciones simbólicas (objetos Action), no un camino en el mapa.

Nota: El proyecto utiliza la librería tkinter para la interfaz gráfica. Si no está disponible, use el modo texto (`-t`) o el modo silencioso (`-q`).

Punto 1: Modelado del Dominio de Planificación (15%)

Antes de poder planificar, es necesario representar formalmente el problema. En este punto usted trabajará directamente con el lenguaje de representación tipo PDDL que provee el proyecto.

Parte 1a: Formalización del dominio (8%)

Abra el archivo planning/domain.py. Allí encontrará el esqueleto de las acciones del robot SAR descritas en la sección anterior, con sus nombres y parámetros definidos, pero con las precondiciones y efectos incompletos.

a) Complete los esquemas de acción en `planning/domain.py` implementando correctamente las precondiciones (lista positiva y negativa) y los efectos (add list y delete list) para cada una de las cinco acciones: `Move` (dado), `PickUp`, `PutDown`, `Rescue` y `SetupSupplies`.
b) Defina los estados objetivo de `SimpleRescueProblem` y `MultiRescueProblem` en `planning/problems.py`. Asegúrese de que el problema captura correctamente todos los fluentes relevantes del mapa.

Tip: Revise cuidadosamente la semántica del operador RESULT: el nuevo estado se forma tomando el estado anterior, eliminando los fluentes del delete list y añadiendo los fluentes del add list. Los fluentes no mencionados permanecen sin cambio.

Parte 1b: Verificación de aplicabilidad (7%)

Una acción es aplicable en un estado si y solo si todos los fluentes positivos de su precondición están presentes en el estado, y todos los fluentes negativos de su precondición están ausentes.

a) Implemente la función `is_applicable(state, action)` en `planning/pddl.py`. Esta función debe retornar `True` si la acción puede ejecutarse en el estado dado.
b) Implemente la función `apply_action(state, action)` que retorna el nuevo estado resultante de ejecutar la acción en el estado dado.
c) Implemente la función `get_applicable_actions(state, domain)` que retorna todas las instancias de acciones (con variables instanciadas a constantes del problema) aplicables en el estado actual. Esta es la función de sucesores que usarán todos sus planificadores.

Punto 2: Búsqueda hacia Adelante (Forward Planning) (20%)

El planificador hacia adelante parte del estado inicial y aplica acciones hacia adelante en el tiempo, explorando el espacio de estados hasta alcanzar un estado que satisfaga el objetivo. Aunque conceptualmente similar a la búsqueda de caminos, ahora los estados son conjuntos de fluentes y las "acciones" son operadores PDDL, lo que genera un espacio de búsqueda mucho más complejo y estructurado.

a) Implemente el algoritmo `forwardBFS` en `planning/planner.py`. Este algoritmo debe explorar el espacio de estados en anchura (BFS) a partir del estado inicial, aplicando acciones aplicables, hasta encontrar un estado que satisfaga el objetivo del problema.
b) Pruebe sus implementaciones sobre los layouts de `layouts/simple/`. Para esta misión modele el problema como `SimpleRescueProblem`.
c) Analice el comportamiento de BFS en términos de número de estados explorados y longitud del plan encontrado en al menos dos layouts diferentes.

Nota importante: El forward planner recibe un objeto Problem que define el estado inicial, la función de sucesores (acciones aplicables y estados resultantes), y la función de prueba de objetivo. Su implementación debe ser genérica y no depender de los detalles específicos del dominio de rescate.

Punto 3: Búsqueda Regresiva (Backward Planning) (20%)

La búsqueda regresiva (o regression search) trabaja en sentido contrario: parte del objetivo y aplica acciones "hacia atrás" para encontrar el estado inicial. Esto reduce el factor de ramificación en muchos dominios, pues solo se consideran acciones que son relevantes para alcanzar los fluentes del objetivo actual.

En la búsqueda regresiva, dado un objetivo g y una acción a, la regresión de g sobre a produce un nuevo objetivo g' tal que si se alcanza g', luego se ejecuta a, se alcanzará g. Formalmente:

$$\text{REGRESS}(g,a) = (g - \text{ADD}(a)) \cup \text{PRECOND}(a) \text{ si } \text{ADD}(a) \cap g \neq \emptyset \text{ y } \text{DEL}(a) \cap g = \emptyset$$

La condición ADD(a) ∩ g ≠ ∅ asegura que la acción es relevante (contribuye al objetivo), y la condición DEL(a) ∩ g = ∅ asegura que no elimina un fluente que ya debe ser verdadero en el objetivo.

a) Implemente la función `regress(goal, action)` en `planning/planner.py`. Esta función recibe un conjunto de fluentes que representan el objetivo actual y una acción, y retorna el objetivo regresado (o None si la acción no es relevante o genera una contradicción).
b) Implemente el algoritmo `backwardSearch` en `planning/planner.py`. Este algoritmo debe explorar el espacio de objetivos hacia atrás a partir del objetivo del problema, hasta encontrar una descripción de objetivo que sea satisfecha por el estado inicial.
c) Pruebe su implementación sobre los layouts de `layouts/simple/`. Modele el problema como `SimpleRescueProblem`.

d) Compare el número de estados (objetivos) explorados por la búsqueda regresiva vs. la búsqueda hacia adelante en los mismos layouts. ¿En qué tipos de problemas espera usted que la búsqueda regresiva sea más eficiente?

Tip: En la búsqueda regresiva, el estado es en realidad una descripción parcial del mundo (un conjunto de fluentes que deben ser verdaderos). El estado inicial del problema se alcanza cuando todos los fluentes en el objetivo regresado son satisfechos por el estado inicial del problema.

Punto 4: Heurísticas para Planificación (20%)

Al igual que en la búsqueda informada, las heurísticas permiten guiar el planificador hacia el objetivo de forma más eficiente. En planificación, las heurísticas más poderosas se derivan automáticamente del dominio mediante relajaciones del problema original. Si el problema relajado es más fácil de resolver, su costo óptimo sirve como estimado del costo del problema original.

Parte 4a: Heurística de ignorar precondiciones (10%)

Esta heurística elimina todas las precondiciones de todas las acciones. Esto significa que cualquier acción puede aplicarse en cualquier estado, y cualquier fluente del objetivo puede alcanzarse en un solo paso. El número de acciones necesarias para satisfacer todos los fluentes del objetivo (resolviendo el problema de cobertura de conjuntos) es el valor heurístico.

a) Implemente la heurística `ignorePreconditionsHeuristic(state, goal, domain)` en `planning/heuristics.py`. Esta función debe estimar el número mínimo de acciones necesarias para satisfacer todos los fluentes del objetivo, ignorando las precondiciones.
b) Demuestre con un ejemplo concreto (usando uno de los layouts del proyecto o creando uno propio) que esta heurística es admisible, es decir, que nunca sobreestima el costo real de la solución.

Parte 4b: Heurística de ignorar listas de borrado (10%)

Esta heurística elimina todos los efectos negativos (delete list) de todas las acciones, creando un problema monótono donde nunca se pierde progreso. En este problema relajado, siempre se puede avanzar hacia el objetivo sin retroceder.

a) Implemente la heurística `ignoreDeleteListsHeuristic(state, goal, domain)` en `planning/heuristics.py`. Para calcular el valor heurístico, resuelva el problema relajado usando hill-climbing: en cada paso, escoja la acción que maximize el número de fluentes del objetivo satisfechos.
b) Implemente el planificador `aStarPlanner(problem, heuristic)` en `planning/planner.py`. Este debe combinar el costo acumulado real con la estimación heurística para guiar la búsqueda forward.
c) Pruebe sus implementaciones sobre los layouts de `layouts/simple/` y compare el número de estados explorados de A* con cada heurística versus forwardBFS sin heurística.
d) ¿Cuál de las dos heurísticas es más informativa en el dominio de rescate? Justifique su respuesta con evidencia empírica.

Punto 5: Planificación Jerárquica de Tareas (HTN) (15%)

Las operaciones de rescate reales no se planifican al nivel de "mover una celda" o "recoger un objeto"; los comandantes piensan en términos de misiones de alto nivel: "rescatar al paciente del edificio norte" o "establecer el puesto médico en el sector B". La planificación jerárquica de tareas (HTN) captura esta estructura natural mediante acciones de alto nivel (HLAs) que se refinan recursivamente en acciones primitivas.

En HTN, una HLA (High-Level Action) tiene uno o más refinamientos posibles, cada uno de los cuales es una secuencia de acciones (que pueden ser HLAs o acciones primitivas). El planificador HTN busca una implementación de la tarea de alto nivel que logre el objetivo.

Parte 5a: Definición de HLAs y refinamientos (10%)

El archivo planning/htn.py contiene el esqueleto de las siguientes HLAs para el dominio de rescate:

- **PrepareSupplies:** Recoger suministros desde la base y llevarlos al puesto médico.
- **ExtractPatient:** Localizar al paciente y transportarlo hasta el puesto médico.
- **FullRescueMission:** Completar una misión de rescate completa (preparar suministros y extraer al paciente).
- **Navigate:** Mover el robot de una celda a otra, posiblemente a través de múltiples celdas intermedias.

Como parte de esta misión, se deben completar los siguientes requisitos:

a) Implemente los refinamientos de cada HLA en `planning/htn.py`. Cada refinamiento debe ser una secuencia de HLAs o acciones primitivas que implementen la tarea de alto nivel. **Recuerde que puede haber múltiples refinamientos posibles para una misma HLA** (por ejemplo, Navigate puede refinarse en distintas secuencias de Move dependiendo de la ruta elegida).

b) Implemente el algoritmo `hierarchicalSearch(problem, hierarchy)` en `planning/htn.py`. Este algoritmo debe buscar en amplitud (BFS) sobre el espacio de planes jerárquicos, reemplazando iterativamente la primera HLA en el plan actual por uno de sus refinamientos, hasta obtener un plan completamente primitivo que alcance el objetivo.

c) Pruebe su implementación sobre los layouts de `layouts/htn/` usando el flag `-m` para activar el modo HTN.

Parte 5b: Misiones múltiples (5%)

Con múltiples pacientes dispersos en el mapa, el orden en que se planifican las misiones puede impactar significativamente el costo total del plan.

d) Extienda su implementación HTN para manejar el problema `MultiRescueProblem`, donde hay múltiples pacientes y suministros. El robot debe rescatar a todos los pacientes. Use la HLA `FullRescueMission` repetidamente para cada paciente.

a) Pruebe su implementación sobre los layouts de `layouts/multi/`. Analice si el orden en que se procesan los pacientes afecta la longitud del plan.

Nota: Algunas instancias de multi-rescate pueden ser computacionalmente costosas. Se recomienda comenzar con layouts pequeños. Si su implementación no puede resolver alguna instancia en tiempo razonable, explique por qué y qué optimizaciones podrían ayudar.

Punto 6: Reflexión sobre el trabajo realizado (10%)

La UNGRD desea documentar las lecciones aprendidas del desarrollo del planeador para mejorar futuros sistemas de respuesta a desastres. Con este fin, le ha solicitado un análisis a partir de su experiencia implementando los algoritmos de planificación. Conteste las siguientes preguntas en un documento PDF:

Parte 6a: Análisis de los algoritmos (6%)

Para cada uno de los puntos anteriores, indique claramente en el contexto del problema de planificación:

- **Complejidad en espacio (Notación O):** ¿Cuánta memoria necesita cada algoritmo en función del tamaño del espacio de estados (número de fluentes n, número de acciones a, profundidad del plan d)?
- **Complejidad en tiempo (Notación O):** ¿Cuántos estados/nodos explora en el peor caso?
- **Completitud:** ¿El algoritmo garantiza encontrar un plan si existe uno?
- **Optimalidad:** ¿El algoritmo garantiza encontrar el plan de menor costo? ¿Bajo qué condiciones?
- **Para los Puntos 4 y 5:** ¿Las heurísticas son admisibles (no sobreestiman)? ¿Son consistentes (satisfacen la desigualdad triangular)? ¿Cuál es la ventaja del planificador HTN sobre el planificador clásico en términos de complejidad?

Se espera que su análisis compare cualitativamente el desempeño de los distintos algoritmos en los layouts de interés, y compare cuantitativamente métricas como la longitud del plan, el número de estados expandidos y el tiempo de ejecución. Presente tablas comparativas cuando sea posible.

Parte 6b: Reflexión sobre el uso de IA (4%)

Escriba una reflexión concisa sobre la co-construcción de la solución final con apoyo de la IA (vea política de uso de la IA más adelante), indicando, por ejemplo, qué aprendizajes obtuvo al ver las correcciones hechas por la IA, si éstas fueron básicas o solo mejoras secundarias; la facilidad o dificultad para crear el prompt adecuado, si la solución propuesta fue fácil de entender (¿usted la podría explicar en una sustentación?). Si no hizo ninguna iteración con IA, indique por qué no le pareció necesario ni útil. Aquí no hay respuestas correctas ni incorrectas. Este punto busca que usted se tome el tiempo de reflexionar sobre el uso de la IA en tareas de programación.

Política de Uso de IA Generativa y presentación en el trabajo a entregar

Para este taller se espera que usted desarrolle una primera versión de la solución de forma completamente autónoma, usando su propio criterio, conocimiento y creatividad antes de recurrir a herramientas de IA. La IA puede emplearse después para mejoras puntuales, refactorización, comentarios de calidad o apoyo en la corrección de errores, pero nunca como sustituto del esfuerzo personal ni como generador principal del código. Use estas herramientas con criterio profesional, asumiendo responsabilidad sobre su proceso de aprendizaje y entendiendo que lo que más valor tiene aquí es el camino que usted recorre para llegar a la solución, no solo el resultado final.

Todos los prompts y las versiones de código generadas con apoyo de IA deben quedar registrados dentro de los archivos del proyecto. Si utiliza IA para refactorizar u optimizar su solución, incluya en el archivo: (1) la versión inicial del código, (2) los prompts que utilizó para refinarla y (3) la versión final del código. Al finalizar, deje como código activo únicamente la versión final y conserve la versión inicial y todos los prompts en forma de comentarios dentro del mismo archivo.

Entrega del trabajo

- Guarde todo su trabajo (incluyendo el documento PDF con la reflexión) en una carpeta comprimida (.zip).
- Suba el trabajo al espacio correspondiente en Bloque Neón a más tardar la fecha y hora indicadas (un solo miembro del grupo debe subir el trabajo).
- Recuerde incluir los nombres y códigos de todos los integrantes de su grupo.

Nota: Al enviar su solución, usted declara que el código entregado en la primera versión de cada punto es de su autoría y que las versiones que utilizan IA fueron la respuesta a los prompts incluidos, que comprende el funcionamiento en todas las versiones y que acepta que este material pueda ser revisado, ejecutado y evaluado por el equipo docente para efectos académicos y de verificación de originalidad.